Contents

Chapter 22: Product Thinking

From Technical Capability to Valuable Product

A technically sophisticated AI system is not necessarily a useful product.

This distinction becomes increasingly important as AI engineering matures. Building an agent that can reason, retrieve documents, call tools, execute workflows, or operate autonomously is an engineering accomplishment. But none of those capabilities establish that someone actually needs the system, will adopt it, or will pay for it.

Product thinking asks a different question:

> **What important human or organizational problem are we solving, and why is AI a particularly effective way to solve it?**

For an AI engineer, this is a critical transition. The objective is no longer simply to optimize model quality or system reliability. It is to optimize **problem–solution fit**.

A useful abstraction is:

$$\text{Value} = f(\text{Problem Importance}, \text{Solution Effectiveness}, \text{Adoption}, \text{Economics})$$

A system can have excellent model performance and still have low value if the underlying problem is unimportant, infrequent, difficult to monetize, or poorly aligned with existing workflows.

This chapter introduces the core concepts required to evaluate AI opportunities before investing heavily in implementation.

---

1. Start With the User Problem

The most common product mistake in AI is to start with the technology.

For example:

> "We can build an agent that autonomously analyzes documents."

That is a capability, not a product.

A product-oriented formulation is:

> "Legal operations teams spend several hours reviewing contracts for a small set of recurring risk conditions. Can we reduce that work while maintaining acceptable accuracy and auditability?"

The second formulation gives us something engineering can optimize.

It identifies:

- a user,
- a workflow,
- a recurring problem,
- a measurable cost,
- a desired outcome,
- and potential constraints.

The distinction is fundamental:

$$\text{Technology} \rightarrow \text{Capability}$$

whereas

$$\text{Problem} \rightarrow \text{Value}$$

The engineering process should therefore proceed roughly as:

$$\boxed{\text{Problem} \rightarrow \text{Workflow} \rightarrow \text{AI Opportunity} \rightarrow \text{Solution}}$$

rather than:

$$\text{LLM} \rightarrow \text{Agent} \rightarrow \text{Find a Use Case}$$

The latter approach frequently produces impressive demonstrations that nobody needs.

---

2. Jobs-to-be-Done

One of the most useful frameworks for product discovery is **Jobs-to-be-Done (JTBD)**.

The central idea is that users do not fundamentally purchase software because they want software. They "hire" a product to accomplish a job.

For example, a manager does not necessarily want:

> "An AI meeting summarization system."

The underlying job might be:

> "After a meeting, I need to know what was decided, who owns each action item, and what I need to follow up on."

The AI system is merely one possible mechanism for accomplishing that job.

A useful formulation is:

$$\text{Job} = \text{Situation} + \text{Desired Outcome} + \text{Constraints}$$

For example:

Situation

A software engineering team has completed a production incident.

Desired outcome

Create an accurate incident report and identify corrective actions.

Constraints

The report must use evidence from logs, tickets, and deployment history and must not invent facts.

This formulation immediately suggests an engineering architecture:

$$\text{Logs+Tickets+Deployment Data} \rightarrow \text{Retrieval} \rightarrow \text{Analysis} \rightarrow \text{Structured Report} \rightarrow \text{Human Review}$$

The product definition and system architecture become connected.

---

3. User Interviews

The engineer's intuition about what users need is frequently wrong.

This is particularly dangerous with AI because AI makes it inexpensive to build prototypes. Engineers can construct sophisticated systems before discovering that the underlying problem is poorly understood.

User interviews are therefore a form of **requirements discovery**.

The objective is not to ask:

> "Would you use an AI agent that does X?"

That question tends to produce unreliable answers.

Instead, investigate existing behavior:

- What are you trying to accomplish?
- Walk me through the last time you did it.
- What happened first?
- What tools did you use?
- Where did you get stuck?
- What takes the most time?
- What errors occur?
- What happens when something goes wrong?
- How frequently does this happen?
- Who else is involved?
- What happens if the problem is not solved?

The key technique is to investigate **observed behavior rather than hypothetical enthusiasm**.

Compare:

> "Would you pay for software that automatically analyzes these reports?"

with:

> "How did you analyze the last report?"

The second question produces evidence about the actual workflow.

---

4. Pain Points

Not every problem is a good product opportunity.

A useful distinction is between **annoyance** and **economic pain**.

Suppose a user spends five minutes each day performing a repetitive task. The task may be annoying, but the economic value of automating it could be small.

Now suppose another user spends four hours every day performing the same kind of task.

The second problem has substantially greater economic significance.

We can think about pain along several dimensions:

$$P = f(\text{Time Cost}, \text{Financial Cost}, \text{Error Cost}, \text{Risk}, \text{Frustration})$$

The most interesting AI opportunities often involve tasks where mistakes are expensive or human effort is unusually costly.

Examples include:

- reviewing large document collections,
- investigating operational incidents,
- processing insurance claims,
- analyzing customer support conversations,
- generating compliance documentation,
- extracting information from unstructured records,
- software maintenance,
- research synthesis.

The important question is not simply:

> "Can AI do this?"

It is:

> **"Is this painful enough that improving it materially changes someone's economics or experience?"**

---

5. Understand the Existing Workflow

AI products rarely replace a single isolated task.

They operate inside workflows.

Consider:

Customer Request → Classification → Investigation → Tool Calls → Decision → Documentation

An AI system might automate only one stage.

Alternatively, an agentic system might coordinate several stages.

This distinction matters because the value of an AI system is determined by its position in the overall workflow.

Suppose a task takes two hours but the AI can automate only ten minutes of it. The product may have limited value.

Conversely, if AI can eliminate most of the workflow while preserving human approval at a critical decision point, the opportunity may be substantial.

A useful engineering representation is:

$$W = s_1, s_2, \ldots, s_n$$

where each s_i is a workflow step.

For each step, measure:

- execution time,
- frequency,
- human involvement,
- error rate,
- information requirements,
- tool dependencies,
- decision complexity,
- consequences of failure.

This produces a **workflow decomposition** that can be mapped directly onto an AI architecture.

---

6. Where AI Actually Creates Leverage

Not every software problem is an AI problem.

Traditional deterministic software is often superior when the task has:

- explicit rules,
- structured inputs,
- deterministic outputs,
- stable requirements,
- well-defined state transitions.

AI becomes particularly interesting when the workflow contains:

- unstructured language,
- images, audio, or video,
- ambiguous inputs,
- large information spaces,
- natural-language interaction,
- complex classification,
- synthesis,
- reasoning,

- knowledge retrieval,
- variable user intent.

This suggests a useful distinction:

$$\text{Software Automation} \neq \text{AI Automation}$$

The question is whether the probabilistic capabilities of AI provide **incremental leverage** over conventional software.

For example:

$$\text{Traditional System} \rightarrow \text{Rules} \rightarrow \text{Structured Output}$$

versus:

$$\text{AI System} \rightarrow \text{Interpretation} \rightarrow \text{Reasoning} \rightarrow \text{Tool Use} \rightarrow \text{Adaptive Output}$$

AI leverage is highest when the latter provides a substantial improvement in economics or user experience.

---

7. Willingness to Pay

Usage and willingness to pay are different variables.

A user might enthusiastically use a free AI tool while having no interest in paying for it.

The fundamental question is:

$$\text{Value Captured by Customer} > \text{Price}$$

For a business application, a simple model is:

$$V = T_s C_h + C_e + C_r + R$$

where:

- T_s = time saved,
- C_h = loaded cost of human labor,
- C_e = avoided error cost,
- C_r = avoided operational risk,
- R = incremental revenue or business value.

The maximum economically rational price is bounded by the value created:

$$P < V$$

This is obviously a simplification. Real pricing depends on alternatives, budgets, procurement processes, switching costs, strategic importance, and competitive dynamics.

Nevertheless, the framework forces a critical question:

Where does the economic value come from?

If that question cannot be answered, the product thesis is weak.

---

8. Adoption Friction

A product can solve a real problem and still fail because adoption is difficult.

AI introduces unusual adoption barriers.

Users may worry about:

- hallucinations,
- privacy,
- security,
- reliability,
- explainability,
- regulatory requirements,
- loss of control,
- workflow disruption,
- model unpredictability.

Therefore:

$$\text{Product Value} \neq \text{Capability}$$

A more realistic formulation is:

$$\text{Realized Value} = \text{Potential Value} \times \text{Adoption Rate} \times \text{Reliability}$$

Suppose an AI system could theoretically save an organization $1 million per year.

If users trust it only 20% of the time, the realized value may be dramatically lower.

This is why human-in-the-loop design is often a product feature rather than merely a safety mechanism.

For example:

$$\text{AI} \rightarrow \text{Recommendation} \rightarrow \text{Human Approval} \rightarrow \text{Execution}$$

may initially create more value than:

$$\text{AI} \rightarrow \text{Autonomous Execution}$$

because the first architecture has a much higher adoption probability.

---

9. Competitive Advantage

An AI feature is not necessarily a durable business.

If a product consists primarily of:

$$\text{Prompt} + \text{LLM API}$$

then competitors may reproduce it quickly.

Durable advantage can instead emerge from:

- proprietary data,
- deep workflow integration,
- distribution,
- user trust,
- domain expertise,
- proprietary evaluation datasets,
- feedback loops,
- network effects,
- switching costs,
- operational infrastructure,
- superior UX,
- accumulated workflow state.

A useful abstraction is:

$$\text{Moat} = f(\text{Data}, \text{Workflow}, \text{Distribution}, \text{Trust}, \text{Integration}, \text{Learning})$$

The strongest AI products often become better because they are embedded in the customer's workflow.

This creates a feedback loop:

$$\text{Usage} \rightarrow \text{Data} \rightarrow \text{Improved System} \rightarrow \text{More Usage}$$

The resulting advantage is considerably stronger than simply having access to a particular foundation model.

---

10. The AI Opportunity Equation

For today's exercise, we will use the following deliberately simple scoring model:

$$\boxed{Opportunity = Pain \times Frequency \times AI\ Leverage}$$

Each dimension can be scored, for example, from 1 to 10.

Pain How costly is the problem?

Consider:

- time,
- money,
- risk,
- errors,
- frustration,
- lost revenue.

Frequency How often does the problem occur?

A problem occurring once per year may be less valuable than a smaller problem occurring hundreds of times per day.

AI Leverage How much does AI change the economics or quality of the task?

A useful scale might be:

Score	AI leverage
1	AI provides little advantage
3	Modest automation
5	Significant productivity improvement
7	Major workflow transformation
10	AI enables something previously impractical

The multiplication is intentional.

A problem with:

$$Pain = 10, \quad Frequency = 1, \quad Leverage = 10$$

has:

$$Opportunity = 100$$

while:

$$Pain = 7, \quad Frequency = 8, \quad Leverage = 8$$

has:

$$Opportunity = 448$$

This illustrates an important product principle:

Large problems are not necessarily large opportunities.

The frequency and AI leverage dimensions matter.

---

11. A More Complete Product Model

The basic equation is useful for discovery, but advanced product analysis should eventually incorporate additional variables.

One possible extension is:

$$Opportunity = P \times F \times L \times W \times A$$

where:

- P = pain,
- F = frequency,
- L = AI leverage,
- W = willingness to pay,
- A = adoption feasibility.

A competitive-adjusted formulation could be:

$$Opportunity^* = \frac{PFLWA}{1 + C}$$

where C represents competitive intensity or difficulty of differentiation.

This is not intended as a scientifically precise metric.

Its purpose is to force explicit reasoning.

Product thinking is fundamentally about making assumptions visible.

---

12. Ten AI Opportunity Candidates

For the exercise, identify ten problems rather than ten AI products.

For example:

1. **Enterprise knowledge retrieval** Employees cannot efficiently locate authoritative information across internal documents and systems.
2. **Software incident investigation** Engineers spend substantial time correlating logs, deployments, tickets, and metrics after production failures.
3. **Contract review** Legal teams manually inspect large volumes of contracts for recurring clauses and risks.
4. **Customer-support resolution** Support agents repeatedly investigate similar problems across documentation, customer history, and operational systems.
5. **Research synthesis** Analysts spend significant time searching, reading, comparing, and synthesizing large document collections.
6. **Compliance evidence collection** Organizations manually assemble evidence required for audits and regulatory processes.
7. **Clinical or administrative documentation** Professionals spend significant time converting conversations and records into structured documentation.
8. **Insurance claim processing** Claims require extracting and reconciling information from heterogeneous documents and communications.
9. **Sales intelligence** Sales teams need to synthesize customer interactions, account information, product data, and competitive intelligence.
10. **Legacy software modernization** Engineers must understand large legacy codebases before safely modifying or migrating them.

The exercise is not to build these systems.

It is to determine which problems deserve further investigation.

---

13. Rank the Opportunities

Create a table like:

Problem	Pain	Frequency	AI Leverage	Opportunity
Enterprise knowledge retrieval	8	10	8	640
Incident investigation	9	7	9	567
Contract review	9	8	8	576
Research synthesis	7	9	9	567
Compliance evidence	8	6	8	384

The numerical ranking is not the conclusion.

It is the beginning of investigation.

A score of 640 does not mean:

> "Build this product."

It means:

> "This hypothesis deserves more attention."

The next stage should involve actual users, workflow observation, competitive research, prototype testing, and economic validation.

---

14. From Opportunity to Product Hypothesis

A high-scoring opportunity should be converted into a testable hypothesis.

For example:

> **Hypothesis:** Software engineering organizations spend significant engineering time investigating production incidents. An AI system that automatically correlates logs, deployments, tickets, and metrics can reduce investigation time by at least 50% while maintaining sufficient evidentiary traceability for engineers to trust its conclusions.

This is much stronger than:

> "Build an AI incident agent."

The first statement contains measurable assumptions.

We can decompose it into:

$$H = (H_p, H_f, H_l, H_a, H_e)$$

where:

- H_p: the problem is sufficiently painful,
- H_f: it occurs frequently,

- H_l: AI provides meaningful leverage,
- H_a: users will adopt the solution,
- H_e: the economics justify the product.

Each hypothesis can then be tested independently.

---

15. Product Thinking Changes Engineering Priorities

This perspective also changes how we allocate engineering effort.

Without product thinking, an AI team might optimize:

$$\text{Model Accuracy} \rightarrow \text{Latency} \rightarrow \text{Cost}$$

With product thinking, the optimization target becomes:

$$\text{User Outcome} \rightarrow \text{Workflow Improvement} \rightarrow \text{Reliability} \rightarrow \text{Economics}$$

Model quality remains important, but it is subordinate to the actual outcome.

For example, improving answer accuracy from 92% to 94% may be irrelevant if users still have to manually perform 80% of the workflow.

Conversely, a modestly accurate model embedded in an excellent workflow may create substantial value.

This is one of the central lessons of AI product engineering:

> **Optimize the system around the user's job, not around the model.**

---

16. Exercise — Find Ten AI Opportunities

Identify **10 real problems** that could plausibly benefit from AI.

For each problem, document:

Problem What is the user trying to accomplish?

Existing Workflow How is the problem solved today?

Pain What makes the current solution expensive, slow, error-prone, or frustrating?

Frequency How often does the problem occur?

AI Leverage What specifically becomes possible or substantially better because of AI?

Willingness to Pay Who receives the economic value, and why might they pay?

Adoption Friction What could prevent users from adopting the solution?

Competitive Advantage What could make a solution difficult to copy?

Then calculate:

$$\boxed{Opportunity = Pain \times Frequency \times AI\ Leverage}$$

Rank the ten opportunities from highest to lowest.

Finally, select the **top three** and write a one-paragraph product hypothesis for each.

---

17. Key Takeaways

1. **Start with problems, not AI capabilities.** An LLM, agent, or RAG system is a technology component, not a product.
2. **Think in terms of jobs-to-be-done.** Understand what outcome the user is actually trying to achieve.
3. **Observe workflows rather than relying on hypothetical user enthusiasm.** Existing behavior is generally more informative than "Would you use this?" interviews.
4. **Pain matters, but pain alone is insufficient.** Frequency and AI leverage determine whether the problem represents a substantial opportunity.
5. **AI leverage is the critical differentiator.** Ask what becomes possible with AI that was previously too expensive, slow, ambiguous, or difficult.
6. **Willingness to pay follows economic value.** Identify who captures the value and how much the improved workflow is worth.
7. **Adoption is part of the product.** Reliability, trust, security, explainability, human approval, and workflow integration can determine whether theoretical value becomes realized value.
8. **A feature is not necessarily a moat.** Durable advantage often comes from proprietary data, workflow integration, distribution, trust, feedback loops, and accumulated domain knowledge.

9. **Quantitative scoring is a prioritization mechanism, not proof.** The opportunity equation helps decide what to investigate; user research and experiments validate the hypothesis.

10. **The ultimate optimization target is user outcome.** The best AI engineering does not maximize model capability in isolation. It maximizes the value delivered by the complete socio-technical system.